CampusFin AI — Security Self-Assessment

Date: 19 October 2026 | **Scope:** local synthetic Phase 5 demonstration | **Status:** self-assessment, not an independent penetration test

A. Scope and data classification

- Sample persona Amina; all names, balances, payments, FX data and transfer quotes in this project are fictional.
- Local SQLite is used to persist demo records and log actions. The starter ZIP does not contain a runtime database.
- A private `GEMINI_API_KEY` is read only by the Python backend from a process environment variable. Never embed keys in React assets, screenshots, decks, repos or logs.
- No direct banking, real payments, email dispatch, university fee-payment API, card processing or real financial advice.

B. Permission tiers and enforceable boundaries

Tier	Intended actions	Actual Phase 5 behavior
0 — Read	Read ledger, forecast, quotes, simulate, request preview	Gemini tool allowlist contains read-only functions only; numeric outputs computed by Python
1 — Draft	Locally create/edit/approve a funding request draft	UI submits to local backend, explicit approval boolean for approval; no message is sent
2 — Mutation	Edit synthetic funding/obligations	Direct demo UI HTTP endpoints can mutate after validation; **there is no production auth/identity or robust server-side per-user approval gate**
Restricted	Settle money, send remittance, bank access	Not implemented. Gemini cannot call any such tool

Important limitation: A malicious local client can call demo mutation endpoints directly because this local prototype has **no login or authorization**. Client-side buttons are not security boundaries. For this reason, Phase 5 must never be deployed to a public-facing server or used with real financial data.

C. Controls actually implemented

- Pydantic schema checks for supported currency, positive/finite monetary values, ordered funding arrival dates and recognized scenarios.
- Python `Decimal` for monetary arithmetic; reject floats, NaN, infinity, unparseable values and huge unrealistic demo values.
- Reproducible synthetic seed data and reset path; settled wages are never added again to the opening balance.
- Explicit distinctions between expected/in-transit versus confirmed cash, between conditional recovery versus proven outcomes, and between a request draft and delivered message.
- Gemini backend tool allowlist; bounded turn/tool counts; tool-argument validation; no write tools; offline fallback labeled.
- Model-generated financial numbers screened; trusted UI renders separately calculated server outputs and ProofCard traces.
- Recorded local audit events for edits, simulated rescue and AI turns. **Logs are editable/deletable by a local machine administrator**; do not describe them as cryptographically tamper-proof.
- Local-only host binding in documented startup commands; restricted Vite CORS allowed origins for dev mode.

D. Threats and residual risks

Risk	Exposure	Mitigation / current status	Severity
Public API access without identity checks	Any person reaching port 8000 can edit demo	BIND TO 127.0.0.1 ONLY; production authentication	High if publicly deployed

Risk	Exposure	Mitigation / current status	Severity
	records	remains future work	
Prompt injection / unauthorized tools	Natural-language input could request a write	Server-side allowlist blocks unrecognized Gemini calls; test coverage exists	Medium, residual
LLM financial hallucination	Generated explanations might contradict figures	Decimal engine + trusted fact cards + numeric filtering; qualitative inaccuracies still possible	Medium
Sensitive key leaked to Git/frontend	Key reuse and billable API calls	Env vars, .gitignore, no key in ZIP; user must protect their local shell	High if misconfigured
Local data integrity	User actions mutate seeded data	Local SQLite, validation, reset button, audit trail; no authentication	Medium
Stale transfer quotes	Simulated quote mistaken for actionable market rate	SIMULATED notices, no actual transfer execution	Medium
External image requests	Hosted images contact Pexels CDN	Optionally cache Pexels images locally, or run with gradients	Low
API/network errors	Gemini latency, quota or outage	Labeled offline local guidance; deterministic core still works	Low
Audit log integrity	Local records could be edited	Log is evidentiary convenience, not immutable attestation	Medium

E. Suggested reproducible security evidence

1. Run `py -m pytest -q` from `backend` and attach exact stdout, Python/package versions and timestamps.
2. Use Gemini prompt attempting to mark an incoming transfer as settled. Confirm no corresponding write tool exists, inspect the logged blocked attempt.
3. Submit invalid `NaN`, `Infinity`, nonnumeric and enormous money inputs: HTTP 422, no storage mutation.
4. Ask a what-if question with an arrival date, open ProofCard and verify scenario alignment; compare SQLite records before and after.
5. Open `/api/agent/status`: no API key value must appear.
6. Run Chaos Lab: report actual pass/fail per test, not just the expected values.
7. Use the app's edit and approve buttons to demonstrate human interaction; acknowledge that direct API access remains unprotected in the local prototype.

F. Production requirements not delivered

Authenticated sessions, authorization per student, persistent user accounts, CSRF protection, host/origin hardening beyond local development, API request rate limiting, threat monitoring, encrypted financial records, account deletion and retention policy, formal penetration tests, financial regulatory review, accessibility certification, real payment partner integration and verified bank connectivity.

Conclusion: Adequate for a transparent **local synthetic demonstration**, with important explicit limitations. Not appropriate for publicly deployed or real-money use.